

Propuesta de Agente Backend Automatizado

1. Resumen

Imagina un **agente como un ingeniero backend automatizado**. El usuario solo le entrega una **fuentes de verdad** (por ejemplo: un schema de base de datos, un archivo JSON, un modelo UML o incluso lenguaje natural). Con eso, el agente diseña una **API REST**, genera la **lógica backend**, agrega funcionalidades típicas de producción, crea **documentación (OpenAPI/Swagger)**, genera **pruebas** y entrega una **API lista para ejecutar**.

Idea central:

Input → modelo de datos → Agente diseña arquitectura + genera código → Output → API funcionando.

2. Fuente de Verdad (Input)

El agente opera con una única fuente principal de verdad que describe entidades, atributos y relaciones. Ejemplos:

- Schema de base de datos (DDL / migraciones / tablas)
- JSON (entidades y relaciones)
- UML (clases, atributos, asociaciones)
- Lenguaje natural (descripción de dominio)

3. Ejemplo de Dominio: E-commerce

El usuario pide una API para un ecommerce con:

- usuarios
- productos
- pedidos

Modelo mental de relaciones:

- Usuario
- Producto
- Pedido
- Pedido -> tiene muchos productos
(En implementaciones reales suele resolverse como una relación N:M con tabla intermedia tipo `pedido_items`.)

4. Flujo de Trabajo del Agente

4.1 Entiende el modelo de datos

- Analiza entidades
- Analiza atributos y tipos
- Analiza relaciones y cardinalidades
- Detecta relaciones N:1, 1:N y N:M (normalmente N:M implica tabla intermedia)

4.2 Diseña la API REST

- Convierte entidades en endpoints
- Define operaciones CRUD y endpoints derivados por relaciones

Ejemplo conceptual de endpoints:

- **GET** /users
- **POST** /users
- **GET** /products
- **POST** /orders

El agente decide también:

- rutas por recurso (/users, /products, /orders)
- rutas para relaciones (por ejemplo, productos de un pedido)
- parámetros de consulta (búsquedas/filtros/paginación) cuando aplique

4.3 Genera la lógica backend

Crea capas típicas de un backend escalable:

- **controladores** (orquestan request/response)
- **servicios** (reglas de negocio)
- **acceso a base de datos** (repositories/queries/ORM)
- **validaciones** (input y consistencia relacional)

4.4 Añade funcionalidades habituales de producción

Incluye automáticamente comportamientos que normalmente haría un desarrollador:

- **autenticación JWT**
- **paginación**
- **manejo de errores consistente** (por códigos HTTP y mensajes)
- **validación de datos** (esquemas y reglas de negocio)
- prácticas de seguridad mínimas (validar payloads, proteger rutas, gestionar secretos vía **.env**)

4.5 Genera documentación automática

- Produce documentación tipo **Swagger / OpenAPI**
- Incluye:
 - esquemas de request/response
 - parámetros de query
 - códigos de error
 - ejemplos de uso

4.6 Genera pruebas

Crea una suite de tests para asegurar el comportamiento:

- **tests unitarios**

- validadores
- servicios (reglas de negocio)
- **tests de endpoints**
 - creación y listado
 - errores por validación
 - flujos con relaciones (por ejemplo, crear pedido con items)

4.7 Entrega una API lista para correr

El usuario recibe algo ejecutable como:

- `npm install`
- `npm start`

Y obtiene una API funcionando.

5. Resultado (Output)

El usuario termina con una API completa que incluye:

- código backend por capas
- modelo de datos implementado
- validaciones
- errores consistentes
- autenticación JWT (si aplica)
- paginación (si aplica)
- documentación OpenAPI/Swagger
- pruebas
- instrucciones de ejecución

6. Esencia del Agente (One-liner)

Input → **modelo de datos**

Agente → **diseña arquitectura + genera código**

Output → **API lista para correr (con docs y tests)**

7. Valor para el Usuario

- Reduce tiempo de implementación inicial
- Disminuye errores repetitivos (CRUD, validaciones, docs, tests)
- Acelera iteración sobre cambios del modelo
- Estandariza buenas prácticas desde el arranque